

Relatório 1º projecto ASA 2024/2025

Grupo: AL043

Aluno(s): Miguel Fabião (109844) e Shimeizi Jin (109947)

Descrição do Problema e da Solução

A nossa resolução do problema consiste na criação de um grafo que contem as ligações entre as linhas. Este é depois analisado aplicando um algoritmo BFS em todas as linhas, calculando o índice de conectividade.

Input e construção do grafo

Os 3 valores numéricos são simplesmente guardados em inteiros. Para os outros valores, criamos um vetor $\langle \text{set} \langle \text{int} \rangle \rangle$ linhas, que guarda linhas e as estações pertencentes a esta linha. E um outro vetor $\langle \text{set} \langle \text{int} \rangle \rangle$ ligações, que corresponde ao grafo que contem as ligações entre as linhas. Algumas verificações são feitas durante a receção dos inputs, num vetor de booleans é confirmado se todas as estações pertencem a uma linha e também é verificado se uma dada linha contem todas as estações.

Lê todas as linhas escritas(m), guarda os dados de acordo com l fazendo operações (l), para encontrar e colocar cada valor dentro de um set temos uma operação logarítmica ($\log(n)$) e para inserir no grafo das linhas ($\log(l)$). A parte de ln vem da confirmação da existência de uma linha que possui todas as estações, linhas(l), número de estações por linha no max (n).

Complexidade total: $O(m.l.(\log(n) + \log(l)) + \ln)$

Overlapping Lines

Para verificar se existe alguma estação contida noutra(visto que esta pode ser ignorada se o mesmo acontecer) criamos esta função. Esta percorre todas as combinações possíveis entre linhas (ignorando as simétricas ex: (1,2), (2,1)) e verifica o esperado. Se alguma for contida noutra, esta é guardada num vetor. Todas as linhas neste vetor, irão ser marcadas como vistas antes de percorrer o BFS, ignorando as.

Percorrer todas as combinações de linhas possíveis (l^2) e a operação de push_back (n).

Complexidade total: $O(l^2.n)$

max_BFS

Esta função é a função principal do nosso algoritmo. Esta corre BFS para todas as linhas, encontrando o índice de conectividade. Esta função faz uso do grafo de ligações entre linhas e do grafo de linhas sobrepostas. Verificamos também se todas as linhas estão ligadas umas com outras, assim, não correndo o risco de haverem 2 ou mais conjuntos de linhas separados.

Percorrer todas as linhas (l) aplicar BFS ($l+m$), sendo l os vertices e m as arestas.

Complexidade total: $O(l.(l+m))$

Complexidade total Teórica: $m \cdot l \cdot (\log(n)) + \log(l) + l^2 \cdot n$

Complexidade espacial

- .O vetor que guarda as estações por linha tem $O(l \cdot n)$.
- .O vetor que guarda as ligações entre estações tem $O(l^2)$.
- .A lista de linhas sobrepostas tem $O(l)$.

Existem outras estruturas de dados auxiliares, no entanto estas apenas existem no contexto de funções, sendo assim apagados quando regressam a main.

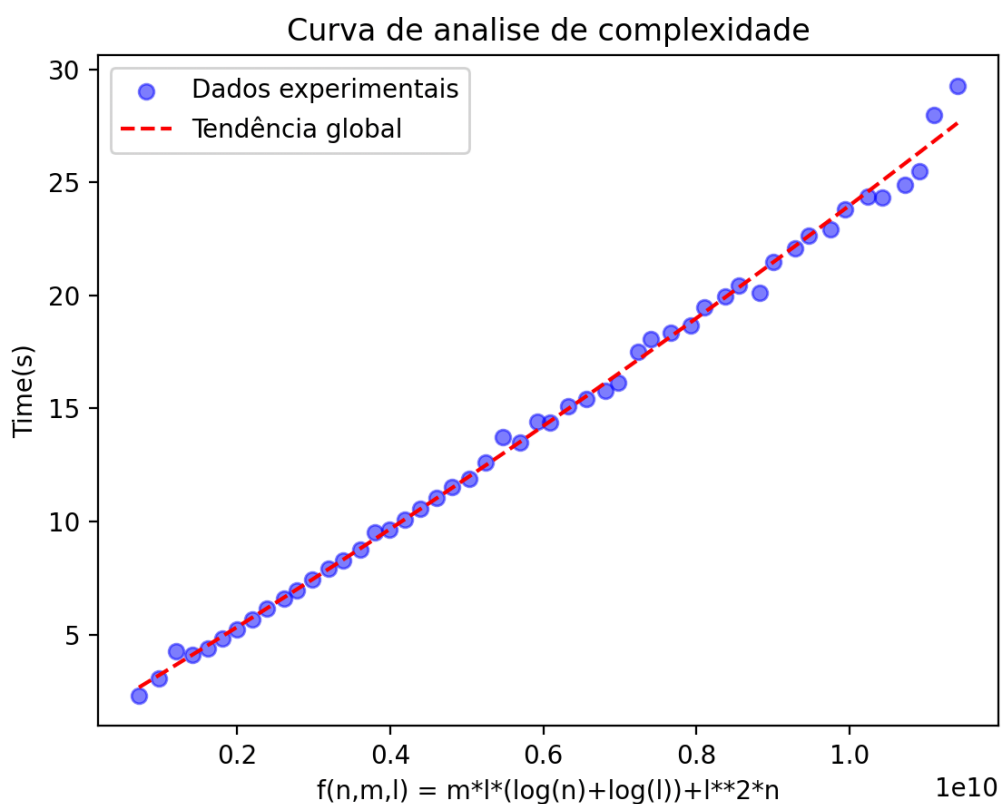
Complexidade Total: $O(l(l+n))$

Análise experimental

Para testar a nossa complexidade obtida na análise teórica, recorremos a um script de python que correu vários testes e fez o gráfico utilizando numpy e matplotlib.

Os testes foram gerados com os seguintes inputs:

```
for l in range(1,51):  
    n = 50000 + i*5000  
    m = 100000 + i*10000  
    l = 100 + 20*log(i)
```



Como podemos ver, o gráfico é linear. Logo confirmamos a nossa complexidade.